

CabRoster

Employee Pickup & Drop Cab Service — Backend

Software Requirements Specification & Development Backlog

Item	Detail
Audience	Students trained in FastAPI, Authentication & Authorisation (JWT, RBAC) and advanced FastAPI (filtering, sorting, pagination)
Timebox	2 hours 30 minutes (individual build)
Tech stack	Python 3.11+, FastAPI, SQLAlchemy 2.x, SQLite, Pydantic v2, JWT (OAuth2 password flow), passlib/bcrypt
Service slots	Morning PICKUP at 06:30 (home → office) and evening DROP at 16:30 (office → home)
Roles	ADMIN, EMPLOYEE, DRIVER
Version / Date	v1.1 — 24 September 2026 (adds Appendix A: Test Contract)
Prepared by	Abinash Kumar Mishra

Contents

Contents.....	2
1. Overview	4
1.1 Problem statement	4
1.2 Learning objectives	4
1.3 Scope.....	4
2. Domain & Business Rules.....	4
2.1 Glossary.....	4
2.2 Slot rules	5
2.3 Business rules (numbered — reference these in code comments and tests)	5
2.4 State machines.....	6
3. Roles & Permissions.....	6
4. Data Model	7
4.1 users.....	7
4.2 cabs	7
4.3 trips	8
4.4 bookings.....	8
4.5 Relationships.....	8
5. API Conventions	9
5.1 General.....	9
5.2 Pagination (all list endpoints)	9
5.3 Sorting.....	9
5.4 Filtering	9
5.5 Error format & status codes.....	9
6. API Endpoints.....	10
6.1 Auth & profile	10
6.2 User management (admin)	10
6.3 Cabs (admin)	10
6.4 Bookings.....	11
6.5 Trips (admin).....	11
6.6 Driver operations	11
6.7 Reports (stretch)	12
7. Sample Requests & Responses	12
8. Development Backlog	12
9. Suggested 2 h 30 min Plan	14
10. Suggested Project Structure	15
11. Acceptance Demo Script.....	15
12. Non-functional Requirements	16

13. Definition of Done.....16

14. Assumptions & Decisions for the Instructor17

Appendix A. Test Contract & Pinned Behaviours.....18

 A.1 Hook points (must exist with these exact names).....18

 A.2 Time rules18

 A.3 Status codes where the spec left a choice.....18

 A.4 Response shapes19

 A.5 Counting, sorting and state rules20

 A.6 Stretch: daily summary definitions (R1)20

 A.7 Running the suites20

1. Overview

1.1 Problem statement

A company provides a free cab service to its employees. Every working day there is exactly one morning **pickup** trip window (cab reaches the employee's home zone and leaves at 06:30 for the office) and one evening **drop** trip window (cab leaves the office at 16:30 for the employees' homes). Today this is managed on spreadsheets and phone calls, which causes over-filled cabs, missed pickups and no record of who actually travelled.

You will build **CabRoster**, a REST backend that lets employees book seats, lets the transport admin group bookings into cab trips, and lets drivers see their passenger list and mark who boarded.

1.2 Learning objectives

By the end of the exercise the student should have demonstrated:

- Modular FastAPI project structure using **APIRouter**, dependencies and settings.
- SQLAlchemy models with relationships, enums and unique constraints.
- JWT authentication with the OAuth2 password flow, password hashing and a **get_current_user** dependency.
- Role-based authorisation (ADMIN / EMPLOYEE / DRIVER) and ownership checks ("a user may only see their own data").
- Business-rule validation (time cut-offs, seat capacity, duplicate prevention) with correct HTTP status codes.
- Filtering, sorting and pagination on list endpoints with a consistent response envelope.

1.3 Scope

In scope (v1)	Out of scope (v1)
User registration & login (JWT)	Real-time GPS tracking / maps
Cab (vehicle) management	Route optimisation algorithm
Seat booking for PICKUP / DROP	Push notifications, SMS, e-mail
Trip creation and passenger assignment by admin	Payments / billing
Driver trip manifest, boarding & trip status	Frontend / mobile app
Filtering, sorting, pagination on all list APIs	Multi-office, multi-shift support
Daily summary report (stretch)	Refresh tokens / SSO (stretch only)

2. Domain & Business Rules

2.1 Glossary

Term	Meaning
Slot	The fixed service window. Enum PICKUP (departs 06:30, home → office) or DROP (departs 16:30, office → home).
Travel date	The calendar date on which the employee wants to travel.
Booking	An employee's request for one seat on a given travel date and slot.
Cab	A vehicle with a unique registration number and a seat capacity (excluding driver).

Term	Meaning
Trip	One cab + one driver running one slot on one date for one zone. A trip carries many bookings.
Zone	Residential area used to group employees in the same cab. Enum: NORTH , SOUTH , EAST , WEST , CENTRAL .
Manifest	The passenger list of a trip shown to the driver.
No-show	Employee booked and was assigned but did not board.

2.2 Slot rules

Rule	PICKUP (06:30)	DROP (16:30)
Direction	Home → Office	Office → Home
Booking opens	Up to 7 days before travel date	Up to 7 days before travel date
Booking cut-off	21:00 on the previous day	14:00 on the same day
Cancellation cut-off	05:30 on the travel date (1 h before)	15:30 on the travel date (1 h before)
Trip can be started by driver	From 06:00 on travel date	From 16:00 on travel date

Time handling (important for testing)

All times are in **Asia/Kolkata (IST)**. Do not call `datetime.now()` directly inside business logic — read the current time through one dependency/function `get_now()` so that tests (and your demo) can override it and simulate "it is 21:05 yesterday".

Slot times and cut-offs must live in `config.py` (e.g. `PICKUP_TIME = "06:30"`), not be hard-coded across files.

2.3 Business rules (numbered — reference these in code comments and tests)

ID	Rule
BR-01	Service runs on working days Monday–Friday only. Bookings for Saturday/Sunday are rejected (422).
BR-02	Travel date must be between today and today + 7 days (inclusive). Past dates are rejected (422).
BR-03	An employee can hold at most one active booking per (travel_date, slot) . A second attempt returns 409 Conflict . A cancelled booking does not block a new one.
BR-04	Booking must be created before the slot's booking cut-off (see 2.2), else 422 with message "Booking window closed".
BR-05	Employee must have <code>home_address</code> and <code>zone</code> set on their profile before booking, else 422 .
BR-06	Employee may cancel their own booking only before the cancellation cut-off. Status becomes CANCELLED (soft cancel — never hard-delete). If it was on a trip, the seat is freed.
BR-07	A cab can be assigned to only one trip per (travel_date, slot) . A driver can drive only one trip per (travel_date, slot) . Violations → 409 .
BR-08	Only active cabs and active users with role DRIVER can be put on a trip.
BR-09	Number of non-cancelled bookings on a trip must never exceed the cab's <code>capacity</code> . Exceeding → 409 "Cab capacity exceeded".

ID	Rule
BR-10	A booking can be added to a trip only if booking.travel_date, booking.slot and booking.zone match the trip, and booking status is BOOKED . On assignment status → ASSIGNED .
BR-11	Bookings can be added to / removed from a trip only while trip status is SCHEDULED .
BR-12	Driver can only see and act on trips where <code>trip.driver_id == current_user.id</code> (otherwise 404 , not 403 , to avoid leaking IDs).
BR-13	Trip status flow: SCHEDULED → IN_PROGRESS → COMPLETED ; SCHEDULED → CANCELLED (admin only). Any other transition → 409 .
BR-14	Driver marks each ASSIGNED booking as BOARDED or NO_SHOW only while trip is IN_PROGRESS .
BR-15	When a trip is COMPLETED , every booking still ASSIGNED becomes NO_SHOW and every BOARDED booking becomes COMPLETED .
BR-16	When admin cancels a trip, all its active bookings go back to BOOKED with <code>trip_id = NULL</code> so they can be re-assigned.
BR-17	A deactivated user cannot log in (401) and a deactivated cab cannot be assigned. Nothing is hard-deleted.

2.4 State machines

Booking status transitions

From	To	Triggered by	Rule
(new)	BOOKED	Employee creates booking	BR-01...05
BOOKED	ASSIGNED	Admin adds booking to a trip	BR-09, BR-10
ASSIGNED	BOOKED	Admin removes from trip, or trip cancelled	BR-11, BR-16
BOOKED / ASSIGNED	CANCELLED	Employee (before cut-off) or admin	BR-06
ASSIGNED	BOARDED / NO_SHOW	Driver during IN_PROGRESS trip	BR-14
BOARDED	COMPLETED	Trip completed	BR-15
ASSIGNED	NO_SHOW	Trip completed while still unmarked	BR-15

Trip status transitions

SCHEDULED → **IN_PROGRESS** (driver start) → **COMPLETED** (driver complete). **SCHEDULED** → **CANCELLED** (admin). Anything else → **409** (BR-13).

3. Roles & Permissions

Three roles are stored in `users.role`. Employees self-register; ADMIN creates DRIVER (and other ADMIN) accounts. The first admin is created by a seed script.

Capability	ADMIN	EMPLOYEE	DRIVER
Register self (becomes EMPLOYEE)	—	✓	—
Login / view own profile	✓	✓	✓
Update own phone / address / zone	✓	✓	✓ (phone only)
Create DRIVER / ADMIN users	✓	✗	✗
List / search all users, activate/deactivate	✓	✗	✗

Capability	ADMIN	EMPLOYEE	DRIVER
Manage cabs (CRUD, deactivate)	✓	X	X
Create / cancel own booking	X	✓	X
List own bookings	—	✓	—
List all bookings (with filters)	✓	X	X
Create trip, assign/remove bookings, cancel trip	✓	X	X
List all trips	✓	X	X
View own assigned trips + manifest	—	X	✓
Start / complete trip, mark boarded / no-show	X	X	✓ (own trips)
View daily summary report (stretch)	✓	X	X

Implement authorisation with a reusable dependency, e.g. `require_roles(Role.ADMIN)`, that returns the current user or raises `403 Forbidden`. Missing/invalid/expired token must return `401 Unauthorized` with header `WWW-Authenticate: Bearer`.

4. Data Model

Four tables are enough for v1. All tables have an integer primary key `id` and `created_at` (UTC timestamp, server default). Use Python `enum.Enum` + SQLAlchemy `Enum` columns.

4.1 users

Column	Type	Constraints / notes
id	int	PK
full_name	str(100)	required, 2–100 chars
email	str(120)	required, unique , stored lower-case, valid e-mail
phone	str(15)	required, 10-digit Indian mobile (regex <code>^[6-9]\d{9}\$</code>)
hashed_password	str	bcrypt hash — never returned in any response
role	enum Role	ADMIN EMPLOYEE DRIVER , default EMPLOYEE
employee_code	str(20)	unique, nullable (required for EMPLOYEE)
home_address	str(255)	nullable (required before booking — BR-05)
zone	enum Zone	nullable (required before booking)
license_no	str(20)	unique, nullable (required for DRIVER)
is_active	bool	default true
created_at	datetime	server default now

4.2 cabs

Column	Type	Constraints / notes
id	int	PK

Column	Type	Constraints / notes
registration_no	str(15)	unique , upper-case, e.g. KA01AB1234 (regex <code>^[A-Z]{2}\d{2}[A-Z]{1,2}\d{4}\$</code>)
model	str(50)	e.g. "Toyota Innova"
capacity	int	seats for passengers, 1–12 (typical 4 or 6)
is_active	bool	default true
created_at	datetime	server default now

4.3 trips

Column	Type	Constraints / notes
id	int	PK
travel_date	date	required
slot	enum Slot	PICKUP DROP
zone	enum Zone	zone this cab serves
cab_id	FK → cabs.id	required
driver_id	FK → users.id	required, user.role must be DRIVER
status	enum TripStatus	SCHEDULED IN_PROGRESS COMPLETED CANCELLED, default SCHEDULED
started_at / completed_at	datetime	nullable, set on transitions
created_at	datetime	server default now

Enforce BR-07 in the service layer (check existing non-cancelled trips). Optional: a DB unique index on (travel_date, slot, cab_id) and (travel_date, slot, driver_id) — note that a cancelled trip must not block a new one, so a service-level check is simpler.

4.4 bookings

Column	Type	Constraints / notes
id	int	PK
employee_id	FK → users.id	required
travel_date	date	required
slot	enum Slot	required
zone	enum Zone	copied from employee profile at booking time
pickup_drop_address	str(255)	copied from <code>home_address</code> at booking time (snapshot)
status	enum BookingStatus	BOOKED ASSIGNED BOARDED NO_SHOW COMPLETED CANCELLED
trip_id	FK → trips.id	nullable; set when assigned
cancelled_at	datetime	nullable
created_at	datetime	server default now

4.5 Relationships

- User (EMPLOYEE) 1 — * Booking
- User (DRIVER) 1 — * Trip
- Cab 1 — * Trip
- Trip 1 — * Booking (a booking belongs to at most one trip)

5. API Conventions

5.1 General

- Base path: `/api/v1`. JSON everywhere except the login endpoint (OAuth2 form data).
- Separate Pydantic schemas for Create, Update (all fields optional) and Read. Read schemas never include `hashed_password`.
- Dates as `YYYY-MM-DD`, datetimes as ISO-8601.
- Swagger UI at `/docs` must work end-to-end using the **Authorize** button.

5.2 Pagination (all list endpoints)

Query param	Type	Default	Rule
page	int	1	≥ 1
size	int	10	1 – 100

Response envelope (generic Pydantic model `Page[T]`):

```
{
  "items": [ ... ],
  "total": 57,
  "page": 2,
  "size": 10,
  "pages": 6
}
```

5.3 Sorting

- Params: `sort_by` (whitelisted per endpoint) and `order = asc | desc` (default `asc` unless stated).
- A `sort_by` value that is not in the whitelist returns `422` — never pass user input straight into `order_by`.

5.4 Filtering

- Exact-match filters (e.g. `status`, `slot`, `zone`) accept enum values; invalid values → `422` automatically via enum typing.
- Range filters use `date_from` / `date_to` (inclusive). `date_from > date_to` → `422`.
- Text search `q` is case-insensitive `ILIKE %q%` on the listed columns.
- All filters are optional and combine with AND.

5.5 Error format & status codes

```
{ "detail": "Cab capacity exceeded" }
```

Code	When
200 / 201	Success / resource created
204	Success with no body (optional for removals)
400	Malformed request not caught by validation

Code	When
401	Missing, invalid or expired token; wrong credentials; inactive user
403	Authenticated but role not allowed
404	Resource not found, or not owned by caller (ownership checks)
409	Conflict: duplicates, capacity, invalid state transition, cab/driver already busy
422	Validation error: bad field, weekend/past date, window closed, profile incomplete

6. API Endpoints

Legend — Auth column: **Public** = no token; otherwise required role(s).

6.1 Auth & profile

#	Method & Path	Auth	Description
A1	POST /auth/register	Public	Register as EMPLOYEE. Body: full_name, email, phone, password (min 8, 1 digit, 1 letter), employee_code, home_address?, zone?. 201 → UserRead. Duplicate email / employee_code → 409.
A2	POST /auth/login	Public	OAuth2 form (username = email, password). Returns {access_token, token_type:"bearer", role}. Token expiry 60 min, claims: sub (user id), role, exp.
A3	GET /auth/me	Any	Returns current user profile.
A4	PATCH /auth/me	Any	Update own phone, home_address, zone (DRIVER: phone only). Cannot change role, email, is_active.

6.2 User management (admin)

#	Method & Path	Auth	Description
U1	POST /users	ADMIN	Create DRIVER or ADMIN (DRIVER requires license_no). 201.
U2	GET /users	ADMIN	List users. Filters: role, zone, is_active, q (full_name, email, employee_code). Sort: full_name, created_at (default), email. Paginated.
U3	GET /users/{id}	ADMIN	Get one user.
U4	PATCH /users/{id}/status	ADMIN	Body {is_active: bool}. Admin cannot deactivate self (409).

6.3 Cabs (admin)

#	Method & Path	Auth	Description
C1	POST /cabs	ADMIN	Create cab. Duplicate registration_no → 409.
C2	GET /cabs	ADMIN	Filters: is_active, min_capacity, q (registration_no, model), available_on + slot (cabs NOT already on a non-cancelled trip for that date/slot). Sort: registration_no (default), capacity, created_at. Paginated.
C3	GET /cabs/{id}	ADMIN	Get one cab.

#	Method & Path	Auth	Description
C4	PATCH /cabs/{id}	ADMIN	Update model, capacity, is_active. Reducing capacity below seats already assigned on a SCHEDULED trip → 409.

6.4 Bookings

#	Method & Path	Auth	Description
B1	POST /bookings	EMPLOYEE	Body {travel_date, slot}. Applies BR-01...BR-05. Copies zone & address from profile. 201 → BookingRead with status BOOKED.
B2	GET /bookings/me	EMPLOYEE	Own bookings. Filters: slot, status, date_from, date_to. Sort: travel_date (default desc), created_at. Paginated. Each item includes trip summary if assigned (cab registration_no, driver name & phone).
B3	GET /bookings/{id}	EMPLOYEE (own) / ADMIN	Booking detail with trip summary. Employee requesting someone else's booking → 404.
B4	PATCH /bookings/{id}/cancel	EMPLOYEE (own) / ADMIN	Cancel (BR-06). Employee is bound by cut-off; admin may cancel any time before trip starts. Already cancelled/completed → 409.
B5	GET /bookings	ADMIN	All bookings. Filters: travel_date, slot, zone, status, employee_id, unassigned (bool: trip_id IS NULL AND status = BOOKED), q (employee name / code). Sort: travel_date, created_at (default), zone. Paginated.

6.5 Trips (admin)

#	Method & Path	Auth	Description
T1	POST /trips	ADMIN	Body {travel_date, slot, zone, cab_id, driver_id}. Validates BR-01, BR-02, BR-07, BR-08. 201.
T2	GET /trips	ADMIN	Filters: travel_date, date_from, date_to, slot, zone, status, driver_id, cab_id. Sort: travel_date (default), created_at. Paginated. Each item shows seats_booked and seats_available.
T3	GET /trips/{id}	ADMIN	Trip detail including cab, driver and passenger list.
T4	POST /trips/{id}/bookings	ADMIN	Body {booking_ids: [int]}. Assigns bookings atomically — if any booking fails BR-09/10/11 the whole request fails and nothing is saved (all-or-nothing). Returns updated trip.
T5	DELETE /trips/{id}/bookings/{booking_id}	ADMIN	Remove a booking from a SCHEDULED trip; booking returns to BOOKED.
T6	PATCH /trips/{id}/cancel	ADMIN	Only from SCHEDULED (BR-13, BR-16).

6.6 Driver operations

#	Method & Path	Auth	Description
D1	GET /driver/trips	DRIVER	Own trips. Filters: travel_date , status , slot . Sort: travel_date (default desc). Paginated.
D2	GET /driver/trips/{id}	DRIVER	Manifest: trip + passengers (name, phone, address, booking status). BR-12.
D3	PATCH /driver/trips/{id}/start	DRIVER	SCHEDULED → IN_PROGRESS; allowed only on travel date from 30 min before slot time. Sets started_at.
D4	PATCH /driver/trips/{id}/bookings/{booking_id}	DRIVER	Body { status : "BOARDED" "NO_SHOW"}. BR-14.
D5	PATCH /driver/trips/{id}/complete	DRIVER	IN_PROGRESS → COMPLETED; applies BR-15. Sets completed_at.

6.7 Reports (stretch)

#	Method & Path	Auth	Description
R1	GET /reports/daily-summary?travel_date=	ADMIN	Per slot: total bookings, assigned, unassigned, cancelled, boarded, no-show, trips, seat utilisation % (boarded ÷ total capacity).

7. Sample Requests & Responses

Create booking — POST /api/v1/bookings

Request: { "travel_date": "2026-09-28", "slot": "PICKUP" }

Response 201:

```
{
  "id": 41, "employee_id": 7, "travel_date": "2026-09-28", "slot": "PICKUP",
  "zone": "EAST", "pickup_drop_address": "12, 5th Cross, Indiranagar",
  "status": "BOOKED", "trip": null, "created_at": "2026-09-25T10:14:03Z"
}
```

Assign bookings — POST /api/v1/trips/9/bookings

Request: { "booking_ids": [41, 42, 45] }

Response 409 (cab of capacity 4 already has 2 seats taken):

```
{ "detail": "Cab capacity exceeded: 2 of 4 seats taken, 3 requested" }
```

List bookings (admin) with filter, sort, page

GET /api/v1/bookings?travel_date=2026-09-28&slot=PICKUP&zone=EAST&unassigned=true
&sort_by=created_at&order=asc&page=1&size=20

8. Development Backlog

Each item is sized to be completed and demoed independently. Priority: **Must** = required to pass; **Should** = expected from strong students; **Stretch** = bonus. Estimates are in minutes for a student who has completed the FastAPI training.

ID	Title	Tasks	Acceptance criteria	Priority	Min
DEV-01	Project setup	Create project skeleton (section 10), <code>config.py</code> with pydantic-settings (DB URL, JWT secret, expiry, slot times, cut-offs, timezone), SQLAlchemy engine/session, <code>get_db</code> dependency, health route <code>GET /health</code> .	App starts with <code>uvicorn app.main:app; /health</code> returns <code>{"status": "ok"}; /docs</code> loads.	Must	8
DEV-02	Models & enums	Implement enums (Role, Zone, Slot, BookingStatus, TripStatus) and the 4 models with relationships; create tables on startup.	SQLite file created with 4 tables; unique constraints on email, employee_code, license_no, registration_no.	Must	10
DEV-03	Security utilities	Password hashing (bcrypt), JWT create/decode, <code>get_current_user</code> , <code>require_roles(*roles)</code> dependency, <code>get_now()</code> time provider.	Invalid/expired token → 401; wrong role → 403; inactive user → 401.	Must	10
DEV-04	Register & login	A1, A2, A3, A4 with schemas and validation.	Duplicate email → 409; weak password → 422; login returns bearer token usable in Swagger Authorize; password hash never returned.	Must	10
DEV-05	Seed script	<code>python -m app.seed</code> creates 1 admin, 2 drivers, 8 employees across 2 zones, 3 cabs (capacities 4, 4, 6).	Script is idempotent (running twice does not duplicate).	Must	5
DEV-06	Common pagination & sorting	Generic <code>Page[T]</code> schema, <code>PaginationParams</code> dependency, helper <code>apply_sort(query, sort_by, order, allowed)</code> , helper <code>paginate(query, page, size)</code> .	Reused by every list endpoint; invalid <code>sort_by</code> → 422; <code>pages</code> computed correctly (ceil).	Must	8
DEV-07	User management	U1–U4.	Filters/sort/pagination work; admin cannot deactivate self; deactivated user cannot log in.	Must	8
DEV-08	Cab management	C1–C4 (excluding <code>available_on</code> filter).	Registration normalised to upper-case; duplicate → 409; capacity 1–12.	Must	8
DEV-09	Create booking	B1 with BR-01...BR-05.	Weekend, past, > 7 days, after cut-off, incomplete profile → 422; duplicate → 409.	Must	12
DEV-10	My bookings & cancel	B2, B3, B4 with BR-06 and ownership checks.	Other employee's booking → 404; cancel after cut-off → 422; cancel twice → 409.	Must	10
DEV-11	Admin booking list	B5 with all filters incl. <code>unassigned</code> .	Combining <code>zone + slot + unassigned=true</code> returns	Must	5

ID	Title	Tasks	Acceptance criteria	Priority	Min
			only matching rows; total reflects filters.		
DEV-12	Create & list trips	T1, T2, T3 with BR-07, BR-08.	Same cab or driver twice for same date/slot → 409; inactive cab / non-driver → 422/409; seats_available correct.	Must	12
DEV-13	Assign / remove bookings	T4 (atomic), T5 with BR-09...BR-11.	Capacity overflow → 409 and no booking changed; zone/date/slot mismatch → 409; status updates to ASSIGNED / back to BOOKED.	Must	12
DEV-14	Driver trip flow	D1–D5 with BR-12...BR-15.	Other driver's trip → 404; mark boarding before start → 409; completion converts ASSIGNED → NO_SHOW and BOARDED → COMPLETED.	Must	12
DEV-15	Cancel trip	T6 with BR-16.	Bookings return to BOOKED with trip_id null; cancelling IN_PROGRESS trip → 409.	Should	5
DEV-16	Cab availability filter	available_on + slot on C2.	Returns only active cabs not on a non-cancelled trip for that date & slot.	Should	5
DEV-17	Daily summary report	R1.	Counts match seed + demo actions; utilisation rounded to 1 decimal.	Stretch	10
DEV-18	Automated tests	pytest + TestClient with in-memory SQLite and get_now override; at least 8 tests covering auth, BR-03, BR-04, BR-09, BR-12.	pytest passes.	Stretch	20
DEV-19	Auto-assign	POST /trips/auto-assign?travel_date=&slot= groups unassigned bookings by zone and fills available cabs/drivers.	No capacity or cab/driver rule is violated; returns summary of trips created.	Stretch	20

Bucket	Items	Estimated time
Must	DEV-01 ... DEV-14	130 min
Should	DEV-15, DEV-16	10 min
Buffer / demo walk-through	—	10 min
Stretch (only if time remains)	DEV-17 ... DEV-19	50 min

9. Suggested 2 h 30 min Plan

Time	Block	Backlog items	Checkpoint
0:00 – 0:08	Read spec, set up project	DEV-01	/health & /docs working
0:08 – 0:43	Models, security, auth, seed	DEV-02 ... DEV-05	Login as seeded admin in Swagger

Time	Block	Backlog items	Checkpoint
0:43 – 0:51	Pagination/sort helpers	DEV-06	Helpers used on users list
0:51 – 1:07	Users & cabs	DEV-07, DEV-08	Admin lists cabs sorted by capacity
1:07 – 1:34	Bookings	DEV-09 ... DEV-11	Employee books PICKUP; duplicate gives 409
1:34 – 1:58	Trips & assignment	DEV-12, DEV-13	Trip with 4 passengers; 5th gives 409
1:58 – 2:10	Driver flow	DEV-14	Full trip completed end-to-end
2:10 – 2:20	Should items	DEV-15, DEV-16	Cancelled trip frees bookings
2:20 – 2:30	Buffer & demo	—	Demo script (section 11) runs clean

10. Suggested Project Structure

```

cabroster/
├── app/
│   ├── main.py           # FastAPI app, include routers, create tables
│   ├── config.py         # Settings: DB, JWT, slot times, cut-offs, TZ
│   ├── database.py       # engine, SessionLocal, Base, get_db
│   ├── models.py         # User, Cab, Trip, Booking + enums
│   ├── schemas/          # user.py, cab.py, booking.py, trip.py, common.py (Page[T])
│   ├── core/
│   │   ├── security.py   # hashing, JWT
│   │   ├── deps.py       # get_current_user, require_roles, get_now, pagination
│   │   └── query.py       # apply_sort, paginate helpers
│   ├── services/         # booking_service.py, trip_service.py (business rules live here)
│   └── routers/          # auth.py, users.py, cabs.py, bookings.py, trips.py, driver.py,
├── reports.py
│   └── seed.py
├── tests/
├── requirements.txt
├── .env.example
└── README.md             # how to run, seeded credentials, demo steps

```

Design guidance

Keep routers thin: they parse input, call a service function and return a schema. Business rules (BR-xx) live in **services/** so they can be tested without HTTP.

Raise **HTTPException** with the status codes from section 5.5, or define small custom exceptions mapped with an exception handler.

11. Acceptance Demo Script

The student runs this sequence in Swagger (or via tests) at the end. Assume "now" is overridden to Friday 25-Sep-2026 10:00 IST and the travel date is Monday 28-Sep-2026.

#	Actor	Action	Expected
1	Admin	Login with seeded credentials	200, token with role ADMIN
2	Employee E1	Register, then try to book without zone/address	422 profile incomplete
3	E1	PATCH /auth/me with address & zone EAST, book PICKUP 28-Sep	201 status BOOKED
4	E1	Book PICKUP 28-Sep again	409 duplicate

#	Actor	Action	Expected
5	E1	Book PICKUP for Saturday 26-Sep	422 non-working day
6	E2...E5	Book PICKUP 28-Sep (zone EAST)	201 each
7	Employee	Access /users	403
8	Admin	GET /bookings?travel_date=2026-09-28&slot=PICKUP&zone=EAST&unassigned=true&sort_by=created_at	5 items, total = 5
9	Admin	Create trip (EAST, cab capacity 4, driver D1)	201 SCHEDULED
10	Admin	Create a second trip with same cab for same date/slot	409
11	Admin	Assign all 5 bookings to trip	409 capacity; none assigned
12	Admin	Assign 4 bookings	200, seats_available = 0
13	Driver D2	GET /driver/trips/{trip_id}	404 (not own trip)
14	Driver D1	Start trip (now = 28-Sep 06:05)	IN_PROGRESS
15	D1	Mark 3 as BOARDED, then complete trip	COMPLETED; 3 COMPLETED + 1 NO_SHOW
16	E1	Cancel a DROP booking at 15:45 on travel date	422 cancellation window closed
17	E1	GET /bookings/me?sort_by=travel_date&order=desc&page=1&size=5	Paginated envelope, newest first

12. Non-functional Requirements

- **Security:** bcrypt hashing; JWT secret from environment (`.env`), never committed; no password or hash in any response or log.
- **Validation:** every input validated with Pydantic (lengths, regex, ranges, enums); e-mails normalised to lower-case.
- **Data integrity:** assignment (T4) is a single DB transaction; no hard deletes.
- **Performance:** list endpoints never return unbounded results (max size 100); avoid N+1 by using `selectinload/joinedload` for trip passengers.
- **Documentation:** every route has `summary`, `tags` and `response_model`; README explains setup and seeded credentials.
- **Code quality:** type hints throughout; no business logic inside router functions beyond orchestration.

13. Definition of Done

1. All **Must** backlog items (DEV-01 ... DEV-14) implemented and meeting their acceptance criteria.
2. Application starts from a clean checkout with `pip install -r requirements.txt, python -m app.seed, uvicorn app.main:app`.
3. Demo script in section 11 runs without unexpected results.
4. Every list endpoint supports the pagination envelope and whitelisted sorting.
5. Correct status codes used for 401 vs 403 vs 404 vs 409 vs 422.
6. Code pushed to the student's repository with README.

14. Assumptions & Decisions for the Instructor

These defaults keep the scope within 2.5 hours. Change any of them before handing out the spec if your batch needs it.

#	Assumption	Alternative if you want more challenge
1	Single office location; one pickup and one drop slot per day.	Multiple shifts / offices as a configurable table.
2	Admin assigns bookings to trips manually.	Make DEV-19 (auto-assign) a Must item.
3	Zones are a fixed enum.	Zones as a table managed by admin.
4	Holidays are not modelled; only weekends are blocked.	Holiday table and check in BR-01.
5	Access token only (60 min).	Refresh token + logout / token blacklist.
6	Tables created with <code>Base.metadata.create_all</code> .	Alembic migrations.
7	Booking snapshot of address; later profile edits do not change existing bookings.	—

Appendix A. Test Contract & Pinned Behaviours

Your code is graded by two automated pytest suites: a **Simple suite** (38 tests, given to you) and a **Full suite** (392 test cases, kept by the instructor). The suites plug into your code at a few fixed points and expect the exact behaviours below. Where sections 2–6 left a choice open, this appendix settles it. **If anything here conflicts with an earlier section, this appendix wins.**

A.1 Hook points (must exist with these exact names)

Import path	What the tests do with it
<code>app.main.app</code>	FastAPI instance under test; routers mounted at <code>/api/v1</code> , health at <code>/health</code> (no prefix).
<code>app.database.Base</code>	<code>Base.metadata.create_all()</code> on a fresh in-memory SQLite DB for every test.
<code>app.database.SessionLocal</code>	Re-bound to the test engine with <code>SessionLocal.configure(bind=...)</code> .
<code>app.database.get_db</code>	Overridden via <code>app.dependency_overrides</code> .
<code>app.core.deps.get_now</code>	Overridden with a fake clock that returns a timezone-aware IST datetime.
<code>app.core.security.hash_password</code>	Used to insert the first ADMIN directly into the DB.
<code>app.models.User</code> , <code>app.models.Role</code>	First ADMIN inserted as <code>User(full_name, email, phone, hashed_password, role=Role.ADMIN, is_active=True)</code> .
<code>app.config.settings</code>	<code>JWT_SECRET_KEY</code> and <code>JWT_ALGORITHM</code> (must be HS256) — tests build their own tokens to check expiry and signatures.

A.2 Time rules

- Every route that needs the time declares `now: datetime = Depends(get_now)`. Never call `datetime.now()` in business logic. If a function reads the time directly, the tests cannot fake the clock and every cut-off test fails.
- JWT `exp` uses the **real** clock `datetime.now(timezone.utc)`, not `get_now()`.
- "Before the cut-off" means `now < cutoff`. At exactly 21:00:00, 14:00:00, 05:30:00 or 15:30:00 the window is **closed**.
- A driver may start a trip only when `now.date() == travel_date` and `now >= departure - 30 min`. At exactly 06:00:00 (PICKUP) or 16:00:00 (DROP) the start is allowed.
- The PICKUP cut-off is 21:00 on the previous **calendar** day. A Monday pickup therefore closes on Sunday at 21:00.

A.3 Status codes where the spec left a choice

Situation	Code
Wrong role on any endpoint, even for an id that does not exist (role is checked before lookup)	403
Role is always read from the DB user, never trusted from the token's <code>role</code> claim	—
POST /users with role EMPLOYEE, or DRIVER without <code>license_no</code>	422
Register body containing <code>role</code> / <code>is_active</code> : ignore them (or reject with 422); the user is never ADMIN	201 / 422

Situation	Code
PATCH /auth/me by DRIVER with home_address or zone	422
PATCH /auth/me with role / email / is_active: fields are ignored	200
GET /cabs with only one of available_on / slot	422
PATCH /cabs capacity below seats on a SCHEDULED trip (completed/cancelled trips don't count)	409
POST /trips: cab or driver id not found	404
POST /trips: cab inactive, or driver not an active DRIVER	422
POST /trips: weekend, past, or more than 7 days ahead (BR-01/02 apply to trips too)	422
POST /trips/{id}/bookings: empty list, or duplicate ids	422
... booking id not found	404
... booking not BOOKED, or date/slot/zone mismatch, or trip not SCHEDULED, or capacity exceeded	409
DELETE /trips/{id}/bookings/{bid}: booking not on this trip	404 (200 or 204 on success)
PATCH /bookings/{id}/cancel: status not BOOKED/ASSIGNED, or its trip already started	409
... employee after cancellation cut-off (admins are not bound by the cut-off)	422
PATCH /driver/trips/{id}/start outside the start window	422
PATCH /driver/trips/{id}/bookings/{bid}: status other than BOARDED / NO_SHOW	422
... booking already marked (not ASSIGNED)	409
Non-integer path id (e.g. /cabs/abc)	422
Any unexpected error	never 500

A.4 Response shapes

Schema	Fields (all required in the JSON)
Token	access_token, token_type ("bearer"), role
UserRead	id, full_name, email, phone, role, employee_code, home_address, zone, license_no, is_active, created_at — no password field of any kind
CabRead	id, registration_no, model, capacity, is_active, created_at
BookingRead	id, employee_id, travel_date, slot, zone, pickup_drop_address, status, trip_id, trip, created_at, cancelled_at — trip is null or {id, status, cab_registration_no, driver_name, driver_phone}
TripRead	id, travel_date, slot, zone, cab_id, driver_id, status, seats_booked, seats_available, started_at, completed_at, created_at
TripDetail	TripRead + cab (CabRead) + driver {id, full_name, phone} + passengers [{booking_id, employee_id, employee_name, employee_phone, pickup_drop_address, status}]
Page[T]	items, total, page, size, pages — pages = ceil(total/size), and 0 when total is 0

Schema	Fields (all required in the JSON)
Assign / remove / cancel trip / start / complete	Return TripDetail. Mark boarding returns BookingRead.

A.5 Counting, sorting and state rules

- `seats_booked` = bookings on the trip whose status is not CANCELLED. Cancelling a booking sets `trip_id = NULL`. `passengers` never lists cancelled bookings.
- Default sort order is ascending, except `GET /bookings/me` and `GET /driver/trips`, which default to `travel_date` **descending**. Ties are always broken by `id` ascending, so paging never repeats or skips a row.
- `unassigned=true` means `trip_id IS NULL AND status = BOOKED` (cancelled bookings are excluded).
- E-mail is compared case-insensitively everywhere: register, duplicate check and login.
- Registration numbers are upper-cased **before** the regex and duplicate checks.

A.6 Stretch: daily summary definitions (R1)

`GET /reports/daily-summary?travel_date=` returns `{"travel_date": ..., "slots": {"PICKUP": {...}, "DROP": {...}}}`. Each slot has: `total_bookings` (all, including cancelled), `assigned` (ASSIGNED), `unassigned` (BOOKED), `cancelled`, `boarded` (BOARDED + COMPLETED), `no_show`, `trips` (non-cancelled), `total_capacity` (sum of cab capacity over non-cancelled trips) and `utilisation_pct` = $\text{round}(\text{boarded} \times 100 \div \text{total_capacity}, 1)$, or 0 when the capacity is 0.

A.7 Running the suites

<code>pytest tests/simple</code>	# 38 basic tests (given to students)
<code>pytest tests/full -m "not stretch"</code>	# 392 cases (grading)
<code>pytest -m stretch</code>	# 5 report tests (bonus)